Proyecto Intermodular 2

Sprint 3 – Proyecto 1

Instrucciones breves de ejecución



Tamara Morillo Gregorio 2º DAMS

ÍNDICE

1. Introducción 2
2. Entidad Tarea 2
3. Repositorio de Tareas 3
4. Controlador de tareas 4
5. Consulta de tareas 5
6. Creación y modificación de tareas 7
7. Eliminación de tareas 8
8. Mejoras en la interfaz de usuario 9
9. Validación de datos 11
10. Seguridad y control de acceso 13
11. Pruebas y verificación de la Definition of Done 14
12. Ejecución de la aplicación 15

1. Introducción

Durante el Sprint 3 se ha ampliado la estructura del proyecto **CatchTask** para incorporar la gestión de tareas académicas. Para ello, se han añadido nuevas clases Java y archivos HTML que permiten integrar esta funcionalidad con las partes de la aplicación desarrolladas anteriormente.

También se han realizado algunos ajustes en los archivos CSS y JavaScript para adaptar la interfaz y mejorar su funcionamiento.

En este documento se explica cómo está organizado el proyecto, cuál es la función de sus principales clases y archivos, y qué pasos hay que seguir para configurar y ejecutar la aplicación.

2. Entidad Tarea

La clase **Tarea.java**, situada en el **paquete entity**, representa las tareas académicas que los usuarios pueden gestionar en **CatchTask**. Esta clase define los datos que se almacenan en la base de datos para cada tarea.

Los **atributos** principales de esta clase son **id, nombre, descripcion, fechaEntrega y terminada**. El atributo id permite distinguir cada tarea y se genera automáticamente. El nombre y la descripción contienen la información de la tarea, fechaEntrega guarda la fecha límite mediante el tipo LocalDate. Por último, el atributo terminada indica si la tarea está completada que, por defecto, se inicializa a false.

Para relacionar la entidad con la base de datos se utilizan varias **anotaciones de JPA**. **@Entity** indica que la clase representa una entidad que se almacena en la base de datos, **@Id** identifica la clave primaria y **@GeneratedValue** indica que el identificador de cada tarea se asigna automáticamente, sin que el usuario tenga que introducirlo manualmente

Además, la clase utiliza **@ManyToOne y @JoinColumn** para establecer relaciones con las entidades Usuario y Asignatura. Cada tarea está asociada a un usuario y a una asignatura. Las columnas **usuario_id y asignatura_id** permiten guardar estas relaciones en la base de datos y, al tener configurado **nullable = false**, no permiten almacenar una tarea sin el usuario o la asignatura correspondientes.

Por último, se utiliza **FetchType.LAZY** para que la información del usuario y de la asignatura se cargue solo cuando sea necesaria, evitando cargar datos que no se van a utilizar.

La clase también incluye un **constructor vacío** y los métodos **getters y setters**, que permiten acceder a sus atributos y modificarlos. De esta forma, Hibernate puede gestionar la persistencia de las tareas en MySQL y facilitar su utilización desde el resto de la aplicación.

Tarea.java

```java
@Entity
public class Tarea {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String descripcion;
    private LocalDate fechaEntrega;
    private boolean terminada = false;

    // Cada tarea pertenece a un usuario
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "usuario_id", nullable = false)
    private Usuario usuario;

    // Cada tarea está asociada a una asignatura
    @ManyToOne(fetch = FetchType.LAZY)
    // No se podrá guardar una tarea sin usuario o sin asignatura.
    @JoinColumn(name = "asignatura_id", nullable = false)
    private Asignatura asignatura;


    // Constructor vacío
    public Tarea() {
    }

    // Getters y Setter
    public Long getId() {
        return id;
```

Figura 1. Clase Tarea.java

3. Repositorio de Tareas

La **interfaz TareaRepository.java** que está situada en el **paquete repository**, se encarga de facilitar el acceso a las tareas almacenadas en la base de datos. Gracias a ella, el resto de la aplicación puede consultar y gestionar las tareas sin necesidad de escribir manualmente las consultas a MySQL.

Esta interfaz extiende de **JpaRepository<Tarea, Long>,** que proporciona operaciones ya preparadas para trabajar con la entidad Tarea, como **guardar, buscar, modificar** y **eliminar** registros.

Además, se ha añadido el método **findByUsuarioId(Long usuarioId),** que permite obtener únicamente las tareas asociadas al usuario indicado. De esta forma, la aplicación puede mostrar a cada usuario sus propias tareas.

TareaRepository.java

```java
package com.proyecto.gestortareas.repository;

import java.util.List;

//Permite acceder a las tareas guardadas en la base de datos
public interface TareaRepository extends JpaRepository<Tarea, Long> {

    // Busca las tareas que pertenecen a un usuario concreto
    List<Tarea> findByUsuarioId(Long usuarioId);

}
```

Figura 2. Repositorio de tareas y método de consulta por usuario.

4. Controlador de tareas

La clase **TareaController.java** situada en el **paquete controller**, se encarga de gestionar las acciones relacionadas con las tareas de **CatchTask**. Recibe las peticiones de las páginas web, utiliza los repositorios para acceder a la base de datos y devuelve la página correspondiente según la operación realizada.

Para gestionar estas peticiones utiliza la anotación **@Controller,** que indica que la clase actúa como controlador web, y @RequestMapping("/tareas"), que establece la ruta principal desde la que se accede a sus diferentes funciones.

Sus funciones principales son mostrar las tareas del usuario y permitir crear nuevas, editar las existentes, guardar los cambios y eliminar las que ya no necesite. También permite cambiar el estado de una tarea para marcarla como terminada o pendiente.

Además, el controlador comprueba que el usuario haya iniciado sesión y que las tareas que intenta modificar o eliminar le pertenezcan. También verifica que la asignatura seleccionada exista y pertenezca al usuario antes de guardar una tarea.

De esta forma, el controlador conecta las páginas de **CatchTask** con la información almacenada en la base de datos y permite gestionar las tareas de forma organizada y segura.

TareaController.java

```java
// Esta clase atenderá las peticiones web.
@Controller
@RequestMapping("/tareas") // Todas las rutas de este controlador empezarán por /tareas.
public class TareaController {

    // Permite acceder a las tareas guardadas en la base de datos.
    private final TareaRepository tareaRepository;

    // Permite consultar las asignaturas disponibles.
    private final AsignaturaRepository asignaturaRepository;

    // Recibe los repositorios necesarios para gestionar las tareas.
    public TareaController(TareaRepository tareaRepository,
                           AsignaturaRepository asignaturaRepository) {
        this.tareaRepository = tareaRepository;
        this.asignaturaRepository = asignaturaRepository;
    }

    // Obtiene el usuario que ha iniciado sesión.
    private Usuario obtenerUsuario(HttpSession session) {
        return (Usuario) session.getAttribute("usuario");
    }
```

Figura 3. Controlador encargado de gestionar las peticiones relacionadas con las tareas de CatchTask.

```java
// Muestra las tareas del usuario conectado.
@GetMapping
public String listarTareas(HttpSession session, Model model) {

    // Comprueba si hay un usuario conectado.
    Usuario usuario = obtenerUsuario(session);

    // Si no hay sesión, vuelve a la página de inicio de sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Busca las tareas que pertenecen al usuario conectado.
    List<Tarea> tareas =
            tareaRepository.findByUsuarioId(usuario.getId());

    // Envía las tareas a la página HTML.
    model.addAttribute("tareas", tareas);

    // Devuelve la página que mostrará el listado de tareas.
    return "tareas";
}
```

Figura 4. Método listar Tareas y comprobación de la sesión del usuario

5. Consulta de tareas

La página **tareas.html**, situada en la **carpeta templates**, es la encargada de mostrar las tareas del usuario en **CatchTask**. Para ello, utiliza Thymeleaf, que permite mostrar los datos recibidos desde el controlador y generar una tarjeta para cada tarea.

En cada tarjeta aparece el **nombre de la tarea**, su **descripción**, la **fecha de entrega**, la **asignatura** a la que pertenece y su **estado actual**, que puede ser pendiente o terminada.

Cada tarea incluye tres opciones: **cambiar su estado, editarla o eliminarla**. En el caso de querer eliminar una tarea, se muestra un mensaje de confirmación antes de realizar la operación para evitar borrados accidentales.

La página también permite añadir nuevas tareas o volver al inicio. Si el usuario todavía no tiene tareas creadas, se muestra un mensaje informativo. Los mensajes de confirmación y error desaparecen automáticamente después de tres segundos.

De esta forma, el usuario puede consultar sus tareas y acceder fácilmente a las principales opciones de gestión desde una misma página.

tareas.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Mis tareas - CatchTask</title>

    <!-- Carga los estilos de la aplicación. -->
    <link rel="stylesheet" th:href="@{/css/estilo.css}">
</head>

<body>

    <!-- Contenedor principal para centrar el contenido. -->
    <div class="contenedor">

        <!-- Tarjeta principal que contiene el listado de tareas. -->
        <div class="tarjeta-listado-tareas">

            <h1>Mis tareas</h1>

            <!-- Muestra el mensaje de confirmación si existe. -->
            <div class="mensaje" th:if="${mensaje}" th:text="${mensaje}"></div>

            <!-- Muestra los errores si existe alguno. -->
            <p class="mensaje"
               th:if="${error != null}"
               th:text="${error}">
            </p>

            <div class="enlaces-tareas">
                <a class="boton-anadir-tarea" th:href="@{/tareas/nueva}">+ Añadir tarea</a>
                <a class="boton-volver-inicio" th:href="@{/inicio}">Volver al inicio</a>
            </div>
```

Figura 5. Estructura inicial de tareas.html y enlaces de navegación.

```
<!-- Mensaje que aparece si todavía no hay tareas. -->
<p th:if="${#lists.isEmpty(tareas)}">
    Todavía no tienes tareas creadas.
</p>

<!-- Rejilla que organiza las tareas en varias columnas. -->
<div class="rejilla-tareas">

    <div class="tarjeta-tarea" th:each="tarea : ${tareas}">

        <div class="contenido-tarea">

            <!-- Nombre de la tarea -->
            <h3 th:text="${tarea.nombre}"></h3>

            <!-- Descripción de la tarea -->
            <p class="descripcion-tarea" th:text="${tarea.descripcion}"></p>

            <!-- Fecha de entrega -->
            <p class="datos-tarea">
                <strong>Fecha de entrega:</strong>
                <span th:text="${tarea.fechaEntrega}"></span>
            </p>

            <!-- Asignatura -->
            <p class="datos-tarea">
                <strong>Asignatura:</strong>
                <span th:text="${tarea.asignatura.nombre}"></span>
            </p>

            <!-- Estado -->
            <p class="datos-tarea">

                <strong>Estado:</strong>
```

Figura 6. Código Thymeleaf para mostrar las tarjetas y la información de cada tarea.

```
            <!-- Botones para realizar acciones sobre la tarea. -->
            <div class="acciones-tarea">

                <!-- Formulario para cambiar el estado de la tarea. -->
                <form th:action="@{/tareas/estado/{id}(id=${tarea.id})}" method="post">
                    <button type="submit"
                            class="boton-estado"
                            th:classappend="${tarea.terminada} ? ' boton-pendiente' : ' boton-terminada'"
                            th:text="${tarea.terminada ? 'Marcar como pendiente' : 'Marcar como terminada'}">
                        Marcar como terminada
                    </button>
                </form>

                <!-- Enlace para editar los datos de la tarea. -->
                <a class="boton-accion enlace-editar"
                   th:href="@{/tareas/editar/{id}(id=${tarea.id})}">
                    Editar
                </a>

                <!-- Formulario para eliminar la tarea, solicitando confirmación. -->
                <form th:action="@{/tareas/eliminar/{id}(id=${tarea.id})}"
                      method="post"
                      onsubmit="return confirm('¿Seguro que quieres eliminar esta tarea?');">
                    <button class="boton-accion boton-eliminar" type="submit">
                        Eliminar
                    </button>
                </form>

            </div> <!-- Cierra el contenedor de los botones de acción. -->
        </div><!-- Cierra la tarjeta individual de la tarea. -->
    </div><!-- Cierra la rejilla que contiene todas las tareas. -->

</div><!-- Cierra la tarjeta principal del listado. -->
</div><!-- Cierra el contenedor principal de la página. -->
```

Figura 7. Opciones de gestión de las tareas

6. Creación y modificación de tareas

La página **formulario-tarea.html**, situada en la **carpeta templates**, contiene el formulario que permite introducir los datos de una tarea. En él se puede indicar el nombre, la descripción, la fecha de entrega y la asignatura a la que pertenece.

El usuario debe completar los campos obligatorios y seleccionar una de sus asignaturas antes de guardar la tarea. Al pulsar el botón “Guardar tarea”, los datos se envían al controlador, que comprueba la información y guarda la tarea en la base de datos.

Este mismo formulario también se utiliza para modificar tareas existentes. En ese caso, el controlador carga los datos de la tarea seleccionada y los muestra en el formulario para que el usuario pueda realizar los cambios.

Además, se muestra un mensaje si se produce algún error y se ofrece la opción Cancelar para volver al listado de tareas sin continuar con el formulario.

De esta forma, se utiliza una misma página para crear y modificar tareas, evitando tener que desarrollar dos formularios diferentes.

formulario-tarea.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Nueva tarea - CatchTask</title>

    <!-- Conecta esta página con la hoja de estilos de la aplicación. -->
    <link rel="stylesheet" th:href="@{/css/estilo.css}">
</head>

<body>

    <main class="contenedor">

        <!-- Tarjeta blanca que contiene el formulario. -->
        <div class="tarjeta">

            <!-- Título del formulario. -->
            <h1 th:text="${accion == 'editar'} ? 'Modificar tarea' : 'Añadir tarea'">
                Añadir tarea
            </h1>

            <!-- Muestra un mensaje si se produce algún error. -->
            <p class="mensaje error"
               th:if="${error != null}"
               th:text="${error}">
            </p>

            <!-- Envía los datos al controlador para guardar la tarea. -->
            <form th:action="${accion == 'editar'}
                    ? @{/tareas/actualizar/{id}(id=${tarea.id})}
                    : @{/tareas/guardar}"
                  th:object="${tarea}"
                  method="post">
```

Figura 8. Configuración del formulario y selección del título según la acción realizada.

```
<!-- Campo para introducir el nombre de la tarea. -->
<label for="nombre">Nombre de la tarea:</label>
<input type="text"
       id="nombre"
       th:field="*{nombre}"
       required
       maxlength="100">

<!-- Campo para explicar qué hay que hacer. -->
<label for="descripcion">Descripción:</label>
<textarea id="descripcion"
          th:field="*{descripcion}"
          rows="4"></textarea>

<!-- Campo para seleccionar la fecha de entrega. -->
<label for="fechaEntrega" class="etiqueta-fecha">Fecha de entrega:</label>
<input type="date"
       id="fechaEntrega"
       name="fechaEntrega"
       th:value="${tarea.fechaEntrega != null ? #temporals.format(tarea.fechaEntrega, 'yyyy-MM-dd') : ''}"
       required>

<!-- Permite elegir una asignatura del usuario conectado. -->
<label for="asignatura">Asignatura:</label>
<select id="asignatura"
        th:field="*{asignatura.id}"
        required>

    <option value="">Selecciona una asignatura</option>

    <!-- Crea una opción por cada asignatura recibida del controlador. -->
    <option th:each="asignatura : ${asignaturas}"
            th:value="${asignatura.id}"
            th:text="${asignatura.nombre}">
    </option>

</select>
```

Figura 9. Campos del formulario para introducir los datos y seleccionar la asignatura.

```
            <!-- Agrupa los botones del formulario para controlar su separación mediante CSS. -->
            <div class="acciones-formulario">

                <!-- Botón para guardar la tarea. -->
                <button type="submit">Guardar tarea</button>

                <!-- Enlace para cancelar y volver al listado de tareas. -->
                <a th:href="@{/tareas}">Cancelar</a>

            </div>
            <!-- Cierra el contenedor de los botones. -->

        </form>
        <!-- Cierra el formulario. -->

    </div>
    <!-- Cierra la tarjeta blanca. -->

</main>
<!-- Cierra el contenedor principal. -->

</body>
</html>
```

Figura 10. Botones para guardar la tarea o cancelar la operación.

7. Eliminación de tareas

CatchTask permite eliminar las tareas que el usuario ya no necesita desde la página "Mis tareas". Para ello, cada tarjeta incluye un botón "Eliminar" que solicita una confirmación antes de continuar, con el objetivo de evitar borrados accidentales.

Cuando el usuario confirma la eliminación, se envía una petición al método eliminarTarea() de TareaController.java. Este método comprueba que haya una sesión iniciada, busca la tarea seleccionada y verifica que pertenezca al usuario conectado.

Si al comprobarlo todo esta correcto, la tarea se elimina de la base de datos mediante tareaRepository.delete(tarea). Después, el usuario vuelve al listado de tareas y recibe un mensaje de confirmación. Si la tarea no existe o no pertenece al usuario, la eliminación no se realiza y se muestra un mensaje de error.

De esta forma, **CatchTask** permite eliminar tareas de manera sencilla y segura, incorporando una confirmación previa y comprobaciones para impedir que un usuario pueda eliminar tareas de otras personas.

```
<!-- Formulario para eliminar la tarea, solicitando confirmación. -->
<form th:action="@{/tareas/eliminar/{id}(id=${tarea.id})}"
      method="post"
      onsubmit="return confirm('¿Seguro que quieres eliminar esta tarea?');">
    <button class="boton-accion boton-eliminar" type="submit">
        Eliminar
    </button>
</form>
```

Figura 11. Formulario de eliminación de tareas con confirmación previa.

TareaController.java

```
253     // Elimina una tarea del usuario que ha iniciado sesión.
254     @PostMapping("/eliminar/{id}")
255     public String eliminarTarea(@PathVariable Long id,
256                                 HttpSession session,
257                                 RedirectAttributes redirectAttributes) {
258
259         // Comprobamos que el usuario haya iniciado sesión.
260         Usuario usuario = obtenerUsuario(session);
261
262         if (usuario == null) {
263             return "redirect:/login";
264         }
265
266         // Buscamos la tarea que queremos eliminar.
267         Tarea tarea = tareaRepository.findById(id).orElse(null);
268
269         // Comprobamos que la tarea exista y pertenezca al usuario actual.
270         if (tarea == null || !tarea.getUsuario().getId().equals(usuario.getId())) {
271             redirectAttributes.addFlashAttribute("error", "No se ha encontrado la tarea.");
272             return "redirect:/tareas";
273         }
274
275         // Eliminamos la tarea de la base de datos.
276         tareaRepository.delete(tarea);
277
278         // Mostramos un mensaje de confirmación.
279         redirectAttributes.addFlashAttribute("mensaje", "Tarea eliminada correctamente.");
280
281         // Volvemos al listado de tareas.
282         return "redirect:/tareas";
283     }
```

Figura 12. Método eliminarTarea() y comprobaciones antes de eliminar una tarea.

8. Mejoras en la interfaz de usuario

Durante el Sprint 3 se realizaron varios ajustes en la interfaz de la aplicación web para mejorar la organización de los elementos y facilitar la navegación por la aplicación.

En la página “Mis tareas”, se modificó la presentación de las tarjetas y la distribución de los botones de acción. De esta manera, las opciones para cambiar el estado, editar y eliminar una tarea quedan mejor organizadas y son más fáciles de localizar.

También se mejoró la distribución de las opciones principales de la página de inicio, separando los accesos a “Mis asignaturas” y “Mis tareas” para que resultara más sencillo navegar por la aplicación.

Por último, se añadió un pequeño código JavaScript para que los mensajes de confirmación y error desaparezcan automáticamente después de tres segundos. Así, el usuario puede recibir información sobre las operaciones realizadas sin que los mensajes permanezcan en pantalla más tiempo del necesario.

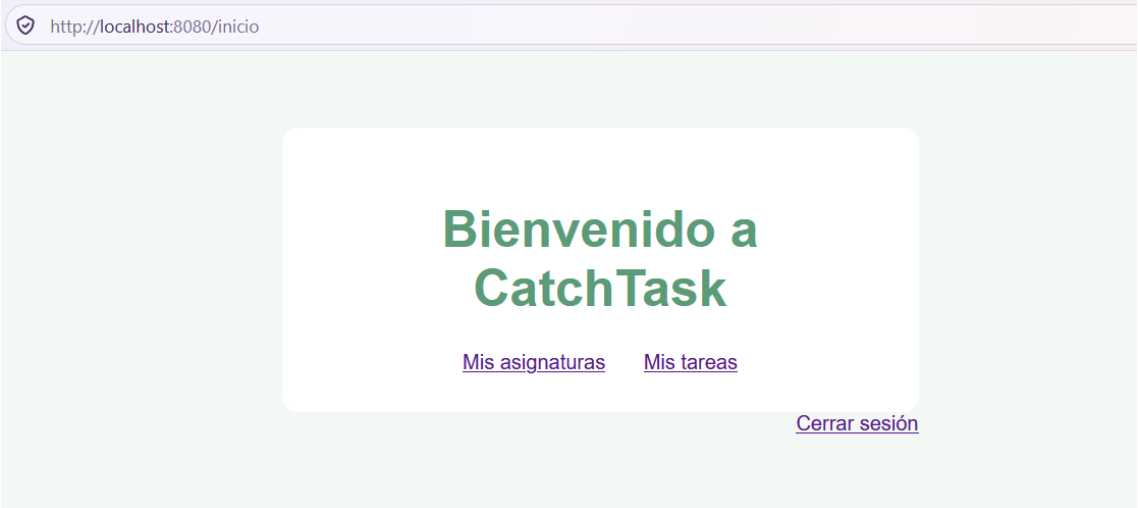


Figura 13. Organización de los accesos principales de la página de inicio.

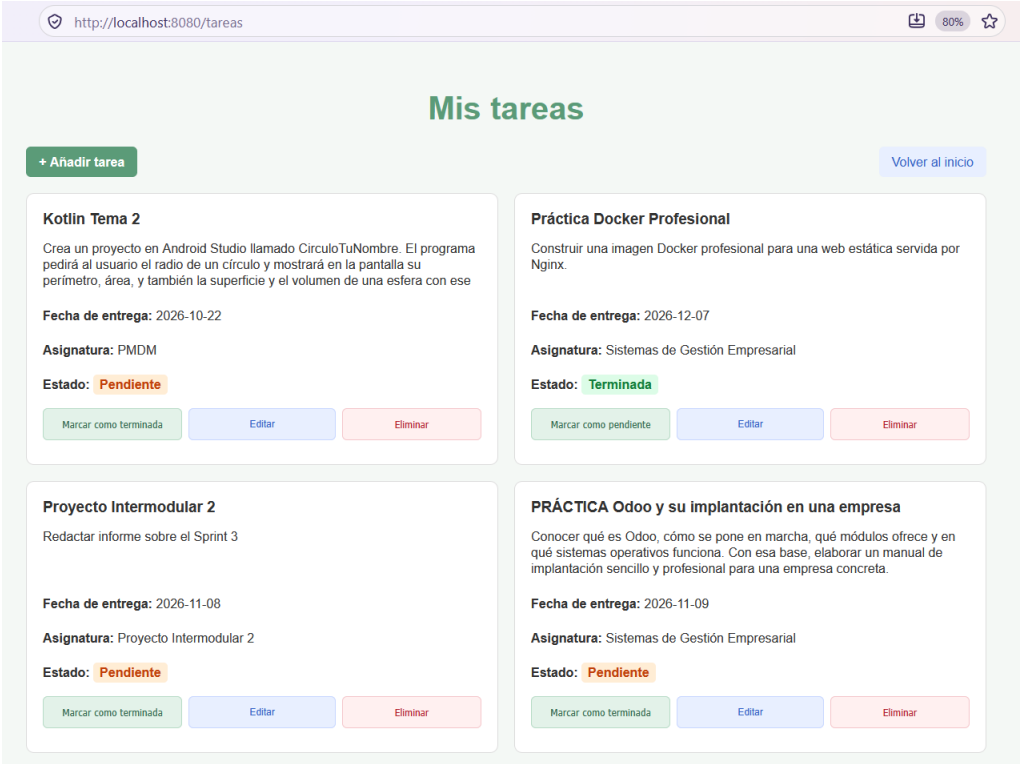


Figura 14. Mejoras en la distribución de las tarjetas y los botones de acción de la página Mis tareas.

```
<script>

    // Espera 3 segundos antes de ejecutar el código
    setTimeout(() => {
        // Busca y elimina todos los elementos con la clase "mensaje"
        document.querySelectorAll(".mensaje").forEach(m => m.remove());
    }, 3000); // 3 segundos
</script>
```

Figura 15. Código JavaScript para eliminar automáticamente los mensajes después de tres segundos.

9. Validación de datos

Durante el desarrollo de la gestión de tareas se incorporaron distintas comprobaciones para evitar que se guarde información incompleta o incorrecta en **CatchTask**.

En el formulario de tareas se utilizan atributos HTML como **required** para indicar los campos que deben completarse obligatoriamente. Por ejemplo, el nombre de la tarea y la fecha de entrega deben tener un valor antes de enviar el formulario. También se establece un límite de 100 caracteres para el nombre.

En el caso de la asignatura, el usuario debe seleccionar una opción del desplegable antes de guardar la tarea. Solo se muestran las asignaturas que tiene registradas en su propia cuenta, evitando que aparezcan las de otros usuarios.

Además, en la clase TareaController.java se realizan comprobaciones adicionales antes de guardar o modificar una tarea. El controlador verifica que el usuario haya iniciado sesión y que la asignatura seleccionada exista y pertenezca a su cuenta.

Si se intenta utilizar una asignatura que no existe o que pertenece a otro usuario, el controlador muestra un mensaje de error y evita guardar los datos. Estas comprobaciones también se realizan al modificar una tarea, junto con la verificación de que la tarea seleccionada pertenezca al usuario conectado.

De esta manera, **CatchTask** combina las validaciones del formulario con las comprobaciones realizadas en Java para reducir errores y proteger la información de cada usuario.

```
<!-- Campo para introducir el nombre de la tarea. -->
<label for="nombre">Nombre de la tarea:</label>
<input type="text"
       id="nombre"
       th:field="*{nombre}"
       required
       maxlength="100">

<!-- Campo para explicar qué hay que hacer. -->
<label for="descripcion">Descripción:</label>
<textarea id="descripcion"
          th:field="*{descripcion}"
          rows="4"></textarea>

<!-- Campo para seleccionar la fecha de entrega. -->
<label for="fechaEntrega" class="etiqueta-fecha">Fecha de entrega:</label>
<input type="date"
       id="fechaEntrega"
       name="fechaEntrega"
       th:value="${tarea.fechaEntrega != null ? #temporals.format(tarea.fechaEntrega, 'yyyy-MM-dd') : ''}"
       required>

<!-- Permite elegir una asignatura del usuario conectado. -->
<label for="asignatura">Asignatura:</label>
<select id="asignatura"
        th:field="*{asignatura.id}"
        required>

    <option value="">Selecciona una asignatura</option>
```

Figura 16. Validaciones HTML aplicadas a los campos del formulario de tareas.

```java
public String guardarTarea(
        @ModelAttribute("tarea") Tarea tarea,
        HttpSession session,
        RedirectAttributes redirectAttributes) {

    // Obtiene el usuario conectado.
    Usuario usuario = obtenerUsuario(session);

    // Impide guardar tareas sin iniciar sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Comprueba que se haya seleccionado una asignatura.
    if (tarea.getAsignatura() == null
            || tarea.getAsignatura().getId() == null) {

        redirectAttributes.addFlashAttribute(
                "error", "Debes seleccionar una asignatura.");

        return "redirect:/tareas/nueva";
    }

    // Busca la asignatura seleccionada en la base de datos.
    Asignatura asignatura = asignaturaRepository
            .findById(tarea.getAsignatura().getId())
            .orElse(null);

    // Comprueba que la asignatura exista y pertenezca al usuario.
    if (asignatura == null
            || !asignatura.getUsuario().getId().equals(usuario.getId())) {

        redirectAttributes.addFlashAttribute(
                "error", "La asignatura seleccionada no es válida.");

        return "redirect:/tareas/nueva";
    }
```

Figura 17. Comprobaciones realizadas antes de guardar una nueva tarea

```java
// Guarda los cambios realizados en una tarea existente.
@PostMapping("/actualizar/{id}")
public String actualizarTarea(@PathVariable Long id,
                              @ModelAttribute("tarea") Tarea datosTarea,
                              Model model,
                              HttpSession session,
                              RedirectAttributes redirectAttributes) {

    // Comprobamos que haya un usuario conectado.
    Usuario usuario = (Usuario) session.getAttribute("usuario");

    if (usuario == null) {
        return "redirect:/login";
    }

    // Buscamos la tarea original para conservar su propietario.
    Tarea tarea = tareaRepository.findById(id).orElse(null);

    // Impedimos modificar tareas que no pertenezcan al usuario.
    if (tarea == null || !tarea.getUsuario().getId().equals(usuario.getId())) {
        return "redirect:/tareas";
    }
```

Figura 18. Comprobaciones de seguridad y validación durante la modificación de tareas

```
// Comprobamos que se haya seleccionado una asignatura válida.
if (datosTarea.getAsignatura() == null
        || datosTarea.getAsignatura().getId() == null) {

    model.addAttribute("error", "Debes seleccionar una asignatura.");
    model.addAttribute("tarea", tarea);
    model.addAttribute("asignaturas",
            asignaturaRepository.findByUsuarioId(usuario.getId()));
    model.addAttribute("accion", "editar");

    return "formulario-tarea";
}

// Comprobamos que la asignatura exista y pertenezca al usuario.
Asignatura asignatura = asignaturaRepository
        .findById(datosTarea.getAsignatura().getId()).orElse(null);

if (asignatura == null
        || !asignatura.getUsuario().getId().equals(usuario.getId())) {

    model.addAttribute("error", "La asignatura seleccionada no es válida.");
    model.addAttribute("tarea", tarea);
    model.addAttribute("asignaturas",
            asignaturaRepository.findByUsuarioId(usuario.getId()));
    model.addAttribute("accion", "editar");

    return "formulario-tarea";
}

// Actualizamos solamente los campos que se pueden modificar.
tarea.setNombre(datosTarea.getNombre());
tarea.setDescripcion(datosTarea.getDescripcion());
if (datosTarea.getFechaEntrega() != null) {
    tarea.setFechaEntrega(datosTarea.getFechaEntrega());
}
tarea.setAsignatura(asignatura);

// Guardamos la tarea actualizada en la base de datos.
tareaRepository.save(tarea);

// Mostramos un mensaje de confirmación al volver al listado.
redirectAttributes.addFlashAttribute("mensaje", "Tarea actualizada correctamente.");

return "redirect:/tareas";
```

Figura 19. Validación de la asignatura y actualización de los datos de la tarea

10. Seguridad y control de acceso

Antes de realizar operaciones relacionadas con las tareas, el controlador comprueba si existe un usuario conectado mediante el método **obtenerUsuario(session)**. Si no hay ningún usuario en la sesión, la aplicación lo redirige a la página de inicio de sesión.

Al mostrar el listado de tareas, se utiliza el método **findByUsuarioId(usuario.getId())** del repositorio. De esta forma, únicamente se recuperan las tareas asociadas al usuario que ha iniciado sesión.

También se realizan comprobaciones al modificar o eliminar una tarea para verificar que pertenece al usuario conectado. Del mismo modo, cuando se crea o modifica una tarea, se comprueba que la asignatura seleccionada exista y pertenezca a ese mismo usuario.

Esto permite mantener separados los datos de cada cuenta y reducir el riesgo de que un usuario pueda consultar o modificar información que no le corresponde.

```java
// Muestra las tareas del usuario conectado.
@GetMapping
public String listarTareas(HttpSession session, Model model) {

    // Comprueba si hay un usuario conectado.
    Usuario usuario = obtenerUsuario(session);

    // Si no hay sesión, vuelve a la página de inicio de sesión.
    if (usuario == null) {
        return "redirect:/login";
    }

    // Busca las tareas que pertenecen al usuario conectado.
    List<Tarea> tareas =
            tareaRepository.findByUsuarioId(usuario.getId());

    // Envía las tareas a la página HTML.
    model.addAttribute("tareas", tareas);

    // Devuelve la página que mostrará el listado de tareas.
    return "tareas";
}
```

Figura 20. Comprobación de sesión y consulta de las tareas del usuario conectado

11. Pruebas y verificación de la Definition of Done

Una vez finalizado el desarrollo de las funcionalidades del Sprint 3, se realizaron diferentes pruebas para comprobar que la gestión de tareas funcionaba correctamente y cumplía los criterios de aceptación establecidos en las **Historias de Usuario**.

Se comprobó la creación de nuevas tareas, verificando que los datos introducidos se guardaran correctamente y que aparecieran en el listado del usuario. También se probaron las opciones de modificación y eliminación, así como el cambio de estado entre pendiente y terminada.

También, se revisaron las validaciones de los campos obligatorios y las comprobaciones de seguridad para evitar que un usuario pudiera gestionar tareas o asignaturas pertenecientes a otras cuentas.

Estas comprobaciones permitieron verificar que las funcionalidades desarrolladas durante el Sprint 3 estaban implementadas y listas para su evaluación, sin errores bloqueantes conocidos.

Prueba realizada	Resultado esperado	
Crear una tarea	Se guarda y aparece en el listado.	**Correcto**
Modificar una tarea	Se actualizan sus datos correctamente.	**Correcto**
Eliminar una tarea	Desaparece del listado después de confirmar.	**Correcto**
Cambiar el estado	La tarea pasa de pendiente a terminada o viceversa.	**Correcto**
Validar campos obligatorios	No se permite enviar el formulario incompleto.	**Correcto**
Comprobar el acceso a tareas de otro usuario	Se impide acceder o modificar tareas ajenas.	**Correcto**
Comprobar la propiedad de las asignaturas	No se permite utilizar asignaturas de otras cuentas.	**Correcto**

12. Ejecución de la aplicación

Actualmente, **CatchTask** se ejecuta de forma local, ya que todavía no se ha realizado su despliegue en un servidor en la nube. El código fuente del proyecto se encuentra alojado en GitHub, desde donde se puede obtener la versión desarrollada.

Para ejecutar la aplicación, es necesario disponer de **Java 21, Eclipse, Maven y MySQL Workbench**, además de tener configurada la base de datos utilizada por el proyecto.

Los **pasos** para poner en marcha el proyecto son los siguientes:

1. Descargar o clonar el repositorio de GitHub en el equipo.

2. Crear la **base de datos** MySQL con el nombre **gestortareas.**

3. Configurar las credenciales de conexión a MySQL mediante las propiedades del archivo application.properties, incluyendo la variable de entorno DB_PASSWORD.

4. Comprobar que el servidor MySQL está iniciado.

5. Abrir el proyecto desde el entorno de desarrollo y ejecutar la clase principal de Spring Boot o iniciar el proyecto mediante Maven.

6. Al iniciar la aplicación, Hibernate se encarga de crear o actualizar automáticamente las tablas necesarias, gracias a la propiedad spring.jpa.hibernate.ddl-auto=update.

7. Abrir el navegador y acceder a **http://localhost:8080** para utilizar **CatchTask**.

Una vez iniciada la aplicación, se puede acceder a las diferentes secciones y comprobar las funcionalidades desarrolladas durante el Sprint 3, como la creación, modificación, eliminación y consulta de tareas.